Taller: Pruebas Unitarias y de Integración

Johan Sebastián Bonilla Gámez

Prof. Jonathan Briceño

Universidad El Bosque

Facultad de Ingeniería

Programa de Ingeniería de Sistemas

Bogotá DC

2025

1. Qué cambió en el código

Se amplió la validación de los datos que recibe la API. La función "validarAvistamiento(body, obtenerHoy)" comprueba que el cuerpo sea un objeto JSON, que los campos especie, lugar, fecha y observador estén presentes y no estén vacíos, que la especie tenga entre 3 y 50 caracteres y que la fecha exista, tenga formato AAAA-MM-DD y no sea posterior al día actual.

La validación está en una función separada de las rutas HTTP. Esto permite probar las reglas directamente, sin iniciar el servidor ni enviar peticiones HTTP. Si la información no cumple las reglas, los endpoints POST /avistamientos y PUT /avistamientos/:id responden con código 400 y un JSON que explica los errores.

Se ampliaron las pruebas en test/api.test.js para cubrir los límites de longitud de la especie, las fechas, cuerpos que no son objetos y los endpoints de la API. Las pruebas de integración usan SQLite en memoria (:memory:), por lo que no modifican el archivo persistente avistamientos.db.

2. Casos de prueba de la validación

Los casos se eligieron usando una partición de equivalencia para separar entradas válidas e inválidas y valores límite, especialmente en la longitud permitida de la especie y en la fecha.

Campo o condición	Valor de prueba	Tipo de caso	Resultado esperado
Cuerpo completo	Objeto con los cuatro campos válidos	Válido	Lista de errores vacía
Campos obligatorios	Faltan campos como lugar, fecha u observador	Inválido	Se informa cada campo faltante
Tipo de cuerpo	null o un valor que no sea un objeto válido	Inválido	Se informa que el cuerpo debe ser un objeto JSON
Longitud de especie	2 caracteres	Límite inferior inválido	Se rechaza la especie
Longitud de especie	3 caracteres	Límite inferior válido	Se acepta la especie
Longitud de especie	50 caracteres	Límite superior válido	Se acepta la especie
Longitud de especie	51 caracteres	Límite superior inválido	Se rechaza la especie
Fecha	Fecha inexistente o formato incorrecto	Inválido	Se informa que la fecha debe ser

			válida y usar el formato requerido
Fecha	Día fijo actual: 2026-10-21	Límite válido	Se acepta la fecha
Fecha	Día siguiente: 2026-10-22	Límite inválido	Se rechaza la fecha futura

Los valores de 2, 3, 50 y 51 caracteres comprueban ambos lados de los límites. Para la fecha se usa un reloj controlado, de modo que la prueba no dependa del día en que se ejecute.

3. Por qué se usan pruebas unitarias y de integración

Las pruebas unitarias verifican directamente validarAvistamiento. Por ejemplo, las pruebas que rechazan una especie de 2 caracteres o aceptan una de 3 caracteres comprueban reglas concretas sin levantar el servidor. Son rápidas y ayudan a localizar errores en la lógica de validación.

Las pruebas de integración envían peticiones HTTP reales a la aplicación. Por ejemplo, la prueba de POST /avistamientos verifica que Express reciba el JSON, ejecute la validación, guarde el registro en SQLite y responda con 201 y el recurso creado. La prueba unitaria del validador, por sí sola, no comprueba toda esa secuencia.

La suite también verifica el flujo de crear, consultar, actualizar y eliminar un registro; los errores por datos incompletos o JSON mal formado; las respuestas 404 para identificadores inexistentes; la ruta inexistente; el endpoint GET /avistamientos/resumen; y el manejo de un error interno que debe responder con 500.

4. Dobles de prueba y aislamiento

Las pruebas de integración abren SQLite en memoria mediante abrirBaseDeDatos(':memory:'). Esto aísla los datos de prueba del archivo persistente utilizado al ejecutar la API normalmente. Sin embargo, SQLite en memoria sigue siendo una base de datos real; no es por sí sola un doble de prueba.

Para controlar las fechas se implementó un stub de reloj. La función de validación recibe una dependencia obtenerHoy, cuyo valor predeterminado devuelve la fecha actual. En las pruebas se pasa relojFijo, que devuelve el 21 de octubre de 2026. Así se comprueba de manera repetible que el día actual se acepta y que el día siguiente se rechaza, sin depender del calendario real.

También se prueba el manejo del error 500 cerrando de forma controlada la base de datos en memoria y haciendo una petición que provoca un fallo de consulta. La prueba comprueba el estado HTTP y el JSON de error. El mensaje statement has been finalized

aparece en consola porque el manejador registra el error con console.error; en esta prueba es intencional y las aserciones pasan.

5. Pruebas que no se incluyeron y por qué

1. **Prueba de carga con cientos o miles de peticiones simultáneas:** El objetivo del taller es comprobar reglas, respuestas HTTP y manejo de errores, no medir rendimiento. Para evaluar capacidad sería más adecuado un ensayo de carga separado.
2. **Pruebas contra una base de datos remota o un servicio externo:** La API trabaja localmente con SQLite y no depende de servicios externos. Añadir una dependencia externa aumentaría la complejidad sin aportar valor a los requisitos actuales.
3. **Prueba de cada posible nombre de ave:** No es práctico enumerar todos los nombres posibles. Las particiones de equivalencia y los valores límite permiten comprobar las reglas de longitud sin probar cada palabra.

Estas decisiones no significan que los escenarios sean inútiles en cualquier proyecto; simplemente quedan fuera del alcance de este taller.

6. Resultados de cobertura

Se ejecutaron las pruebas con el comando:

powershell node --test --experimental-test-coverage

Lo que se obtuvo fue:

Archivo	Líneas	Ramas	Funciones
src/app.js	100,00 %	97,87 %	100,00 %
src/db.js	100,00 %	100,00 %	100,00 %
Total	**100,00 %**	**97,96 %**	**100,00 %**

La suite ejecutó 18 pruebas: 18 aprobadas y 0 fallidas. Todas las líneas y funciones contabilizadas se ejecutaron. La cobertura de ramas quedó en 97,96 %, por lo que todavía hay alguna alternativa de decisión sin cubrir.

La cobertura es un indicador de qué parte del código se ejecutó, pero no demuestra por sí sola que todas las reglas estén bien verificadas. También importa que cada prueba tenga aserciones relevantes y compruebe el resultado esperado. No se considera necesario agregar pruebas únicamente para perseguir el 100 % de ramas si no aportan valor al comportamiento que se desea verificar.

Conclusión

Las 18 pruebas verifican reglas de validación, valores límite, operaciones principales de los endpoints, respuestas de error y el resumen por especie. La separación de

validarAvistamiento facilita las pruebas unitarias, mientras que las peticiones HTTP comprueban la integración entre Express y SQLite. El stub de reloj hace repetibles las pruebas de fechas y la prueba del error interno comprueba la respuesta 500. La cobertura final es de 100 % en líneas y funciones, y 97,96 % en ramas.